

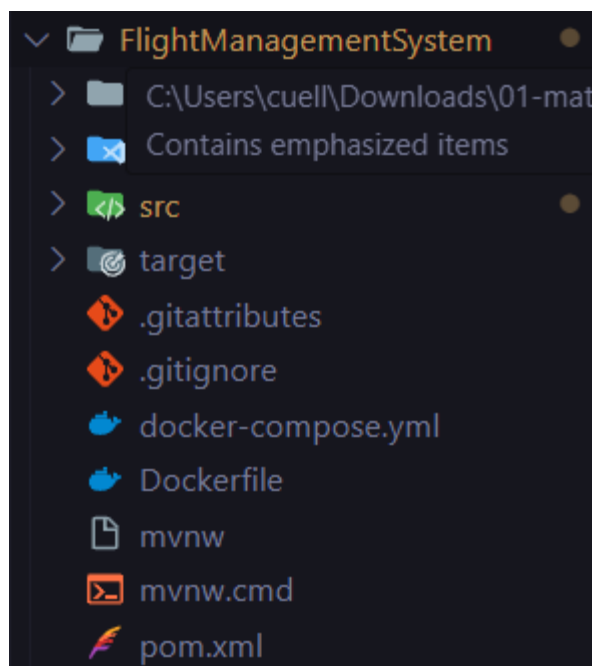
MANUAL CI/CD DOCKER

1) Requisitos previos

- Tener **Docker Desktop** (Windows/Mac/Linux) instalado y corriendo.
- Git / Editor.

2) Estructura de proyecto recomendada

Colocar los archivos en la raíz de tu proyecto (donde está pom.xml):



3) Dockerfile

Crea Dockerfile en la raíz:

```
Dockerfile X
01-material > Backend > FlightManagementSystem > Dockerfile
1 FROM maven:3.9.6-eclipse-temurin-17 AS build
2 WORKDIR /app
3 COPY pom.xml .
4 RUN mvn dependency:go-offline
5 COPY src ./src
6 RUN mvn clean package -DskipTests
7
8 FROM eclipse-temurin:17-jdk-jammy
9 WORKDIR /app
10 COPY --from=build /app/target/*.jar app.jar
11 ENV SPRING_DATASOURCE_URL=jdbc:postgresql://localhost:5432/flight_management?connectTimeout=10&sslmode=prefer
12 ENV SPRING_DATASOURCE_USERNAME=postgres
13 ENV SPRING_DATASOURCE_PASSWORD=12345678
14 EXPOSE 5432
15 ENTRYPOINT ["java", "-jar", "app.jar"]
16
```

Qué hace: compila el .jar dentro de una imagen Maven (no necesitas Maven en tu PC) y luego ejecuta con JRE más ligero.

4) docker-compose.yml (backend + base de datos)

Este archivo orquesta varios contenedores

```
1  services:
2    postgres:
3      container_name: serve-postgres-sena
4      image: postgres:16
5      restart: always
6      environment:
7        POSTGRES_PASSWORD: n7cuellar
8        POSTGRES_USER: postgres
9        POSTGRES_DB: flight_management
10     ports:
11       - "5432:5432"
12     networks:
13       - network_local_server
14     volumes:
15       - postgres_data:/var/lib/postgresql/data
16       - "/home/ubuntu/db_backups:/backups/postgres"
17     healthcheck:
18       test: ["CMD-SHELL", "pg_isready -U postgres"]
19       interval: 30s
20       timeout: 10s
21       retries: 5
22     labels:
23       - com.sena.group=databases
24
25  backend:
26     container_name: backend-flight-management
27     build:
28       context: .
29       dockerfile: Dockerfile
30     restart: always
31     depends_on:
32       postgres:
33         condition: service_healthy
34     environment:
35       SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/flight_management
36       SPRING_DATASOURCE_USERNAME: postgres
37       SPRING_DATASOURCE_PASSWORD: n7cuellar
38     ports:
39       - "8081:8081"
40     networks:
41       - network_local_server
42
43  volumes:
44     postgres_data:
45       driver: local
46
47  networks:
48     network_local_server:
49       driver: bridge
50       name: network_local_server
```

- **db** → levanta PostgreSQL con usuario, contraseña y base de datos inicial.
- **app** → construye tu Backend a partir del Dockerfile.
- **depends_on** → garantiza que la DB esté lista antes de arrancar el backend.
- **volumes** → guarda los datos de la base fuera del contenedor para que no se pierdan al reiniciar.

5) Construir y levantar

Ejecuta desde la carpeta donde está docker-compose.yml:

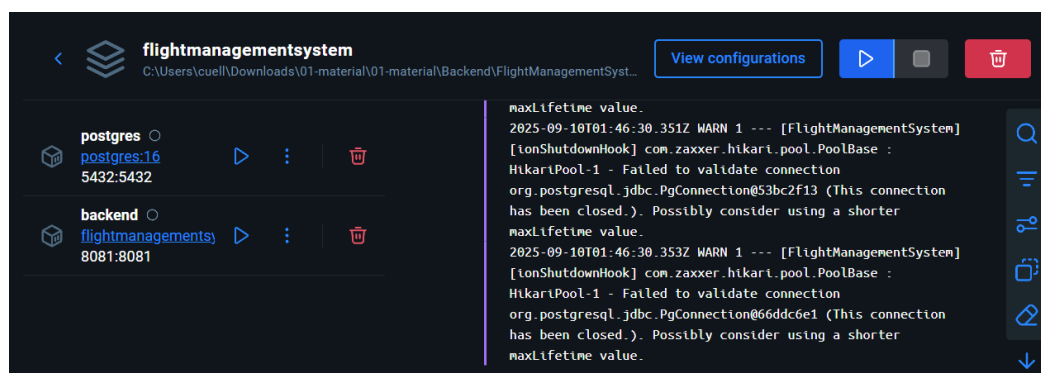
```
PS C:\Users\cuell\Downloads\01-material\01-material\Backend\FlightManagementSystem> docker-compose up -d --build
Compose can now delegate builds to bake for better performance.
To do so, set COMPOSE_BAKE=true.
[+] Building 279.2s (18/18) FINISHED
```

- **-d** → modo “detached” (segundo plano).
- **--build** → fuerza la reconstrucción de la imagen del Backend.
- Mensaje de finalización:

```
[+] Running 5/5
✓ backend Built
=> => unpacking to docker.io/library/flightmanagementsystem-backend:latest
=> [backend] resolving provenance for metadata file
[+] Running 5/5
✓ backend Built 0.0s
✓ Network network_local_server Created 0.2s
✓ Volume "flightmanagementsystem_postgres_data" Created
✓ Container serve-postgres-sena Healthy
✓ Container backend-flight-management Started
```

6) Abrir Docker Desktop

Se abre el Docker Desktop



7) Añadir Esquemas en la terminal

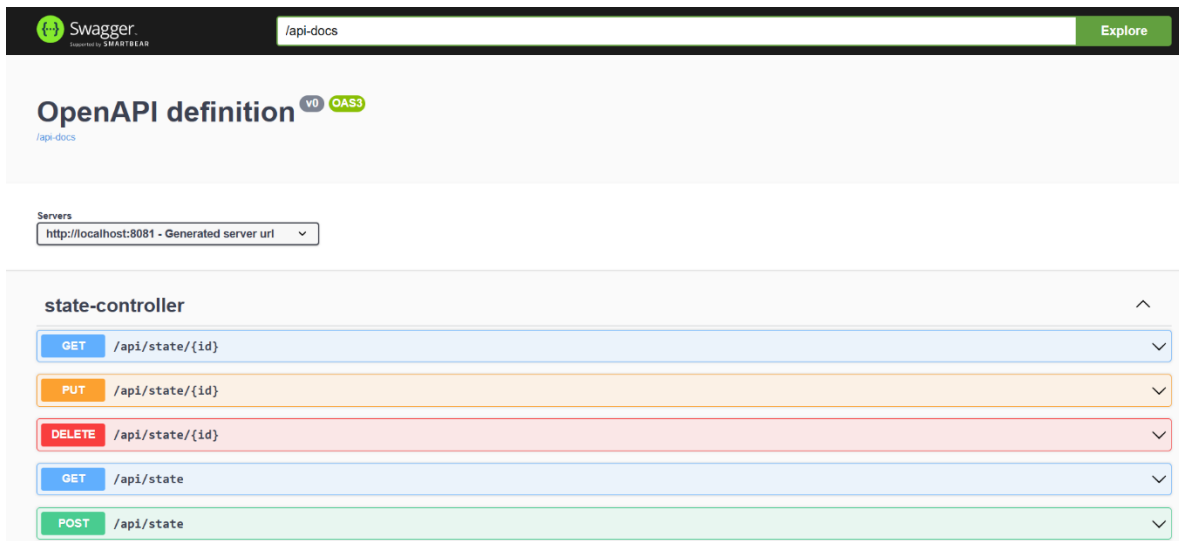
Se crea los esquemas desde la terminal

```
PS C:\Users\cuell> docker exec -it serve-postgres-sena psql -U postgres -d flight_management
psql (16.10 (Debian 16.10-1.pgdg13+1))
Type "help" for help.

flight_management=# CREATE SCHEMA aircraft_management;
CREATE SCHEMA flight;
CREATE SCHEMA geolocation;
CREATE SCHEMA human_resources;
CREATE SCHEMA infrastructure;
CREATE SCHEMA
CREATE SCHEMA
CREATE SCHEMA
CREATE SCHEMA
CREATE SCHEMA
```

8) Abrir Swagger

Se abre el Swagger y se prueba los Endpoints



The screenshot displays the Swagger UI interface. At the top, there's a header with the Swagger logo and a search bar containing '/api-docs'. Below this, the main heading reads 'OpenAPI definition' with a 'v0' tag and an 'OAS3' badge. A 'Servers' section shows a dropdown menu with 'http://localhost:8081 - Generated server url'. The main content area is titled 'state-controller' and lists five API endpoints, each with a colored button indicating the HTTP method and a dropdown arrow for details:

- GET** /api/state/{id}
- PUT** /api/state/{id}
- DELETE** /api/state/{id}
- GET** /api/state
- POST** /api/state